

Paytm Wallet R2 — One-Page Engineering Design

1. Problem and invariants

The service implements four operations: **get-or-create wallet**, **read wallet**, **create P2P transfer**, and **read transfer status**. I treated it primarily as a concurrency/correctness problem rather than a REST problem. The invariants are:

- **Conservation:** money moved between wallets must not change the system total.
- **No overdraft:** `balance_paise` must never be negative.
- **Exactly-once financial effect per idempotency key.**
- **Race-free wallet creation:** one authenticated user must end up with one wallet even under concurrent requests.

All monetary values use Java `long` and PostgreSQL `BIGINT` in integer paise; no floating-point arithmetic is used.

2. Data model and database-enforced guarantees

```
wallets
id UUID PK, user_key VARCHAR UNIQUE NOT NULL, balance_paise BIGINT NOT NULL,
timestamps, CHECK (balance_paise >= 0).
```

```
transfers
id UUID PK, idempotency_key VARCHAR UNIQUE NOT NULL, caller_user_key,
from_wallet_id, to_wallet_id, amount_paise BIGINT, status, decline_reason,
timestamps, wallet FKs, CHECK (amount_paise > 0),
CHECK (from_wallet_id <> to_wallet_id).
```

The database is part of the correctness model. Java orchestrates the flow, while PostgreSQL owns uniqueness, atomicity, row locking, and final safety constraints.

3. Simplest-correct concurrency mechanism

Wallet creation — no JVM lock and no singleton

For `POST /wallets`, I did **not** use `synchronized`, an in-memory singleton, or a read-then-insert sequence. Those approaches fail once more than one application instance exists.

The database owns uniqueness:

```
50 concurrent requests
```

```
|
```

```

      v
INSERT wallet ... ON CONFLICT(user_key) DO NOTHING
      |
      v
UNIQUE(user_key)
      |
      v
exactly one row exists

```

If one request wins the insert, the others read the same existing wallet. This is simpler and stronger than an application lock because the guarantee survives multiple JVMs and restarts.

Transfer — row-level pessimistic locking

A transfer runs in **one PostgreSQL transaction**. Only the two participating wallet rows are locked; the whole table is never locked.

```

BEGIN
|
+--> check committed idempotent result
|
+--> sort the two wallet IDs deterministically
|
+--> SELECT wallet #1 ... FOR UPDATE
|
+--> SELECT wallet #2 ... FOR UPDATE
|
+--> claim idempotency key
|
+--> conditional debit: balance >= amount
|
+--> credit destination
|
+--> persist SUCCEEDED / DECLINED
|
COMMIT

```

The lock order is based on a stable ordering of wallet IDs, **not transfer direction**. Therefore $A \rightarrow B$ and $B \rightarrow A$ both acquire the same rows in the same order.

BAD:	CHOSEN:
T1 locks A	both transactions
T2 locks B	lock min(A,B)
T1 waits B	then max(A,B)

```
T2 waits A
=> deadlock risk          => deterministic order
```

This is the same class of concurrency problem seen in inventory reservation, seat booking, account transfers, and other systems where many requests compete to mutate a small set of shared rows.

The debit is also conditional:

```
UPDATE wallets
SET balance_paise = balance_paise - ?
WHERE id = ?
  AND balance_paise >= ?
```

and PostgreSQL additionally enforces:

```
CHECK(balance_paise >= 0)
```

Locking prevents conflicting writers from racing; the conditional update and DB constraint provide defense in depth.

4. Design patterns actually used

The design is intentionally small and uses only patterns that solve a requirement:

- **Layered architecture:** Controller → Service → Repository → PostgreSQL.
- **Repository pattern:** SQL and persistence details stay out of controllers and business logic.
- **Transaction Script / Unit of Work:** `TransferService` coordinates the complete financial mutation inside one `@Transactional` boundary.
- **Database-backed idempotency:** the unique key is durable and shared across instances.
- **Constraint-based invariants:** database uniqueness/check constraints remain valid even if application code has a defect.
- **Filter/interceptor approach:** authentication and correlation IDs are handled outside business logic.

5. Idempotency

Idempotency lives in PostgreSQL through `UNIQUE(idempotency_key)` and is committed in the **same transaction as the money movement**.

- Same key + same request → return the original result.
- Same key + different request → `409 Conflict`.
- Commit succeeds but HTTP response is lost → retry returns the original transfer, not another debit.
- A declined transfer is also persisted, so replaying it remains deterministic.

This gives **exactly one financial effect for a given idempotency key**.

6. Why this mechanism instead of heavier alternatives

I considered stronger or more distributed approaches but rejected them for this bounded problem:

- **Global `SERIALIZABLE`**: correct, but introduces serialization failures/retries and broader contention when explicit row locks are sufficient.
- **Redis/distributed locks**: extra infrastructure and failure modes; PostgreSQL already owns the authoritative data.
- **Kafka/eventual consistency**: useful for downstream events, but unnecessary for synchronous two-wallet mutation where debit and credit must commit atomically.
- **Microservices/distributed transactions**: would turn one local ACID transaction into a distributed consistency problem.
- **Java `synchronized`**: only protects one JVM.
- **Naive read-modify-write**: vulnerable to lost updates.

The principle was: **use the smallest mechanism that directly enforces the required invariant.**

7. Concurrency and stress validation

I validated the design with integration tests and an HTTP burst suite against the deployed service:

- **Gate 1**: 50 concurrent get-or-create requests → exactly **1 wallet**.
- **Gate 2**: 30 concurrent requests with the same idempotency key → **1 transfer / 1 debit / 1 credit**, identical replay responses, changed payload returns `409`.
- **Gate 3**: 300 transfers at **40 concurrent workers**, including opposite-direction contention and intentional overdrafts → total balance `300000 → 300000`, `284` succeeded, `16` declined, **no negative balances**.

The application pool was configured to 30 DB connections. Running 40 concurrent client workers intentionally exercised real database contention and connection-pool pressure rather than only a single-threaded happy path.

A deployment issue also surfaced: Railway initially ran in US West while Neon ran in Singapore. Under row-lock contention, network latency extended transaction duration and therefore extended lock-hold time, eventually exhausting the connection pool. Moving the application to **Singapore**, next to the database, removed that latency amplification and the complete burst passed.

This reinforced an important production rule: **network round trips inside a database transaction directly increase how long locks are held.**

8. Consistency vs availability

For financial mutations I choose **consistency over availability**. If PostgreSQL is unavailable or a safe transaction cannot be established, the API fails rather than acknowledging an uncertain payment or queueing an unknown financial state.

For a wallet, **temporarily unavailable is safer than possibly debited twice.**

9. Deployment, observability, and cost

The application is packaged as a multi-stage, non-root Docker image and deployed on Railway. PostgreSQL is hosted on Neon, with both application and database placed in Singapore. Flyway owns schema migration at application startup.

Observability includes structured JSON logs with `correlation_id` and domain events such as:

```
transfer_created
transfer_debited
transfer_credited
transfer_succeeded
transfer_declined
idempotent_replay_hit
```

Micrometer/Prometheus metrics expose request rate, latency, HTTP errors, transfer outcomes, idempotency replays/conflicts, and Hikari connection-pool state.

The deployment was deliberately cost-conscious: one stateless application service and one managed PostgreSQL database, with no Kafka, Redis, or other always-on infrastructure. Railway free/trial resources and Neon free-tier resources were sufficient for the exercise.

Actual exercise cost: ₹0.

10. AI usage — directed vs decided

AI was used as an engineering assistant for **initial boilerplate generation, technology comparison, design exploration, test-case generation, concurrency/stress-test creation, code review, debugging suggestions, and documentation drafting.**

The **problem decomposition, design philosophy, code ownership, acceptance/rejection of alternatives, debugging decisions, deployment choices, and final validation were mine.** I reviewed and modified the implementation, executed the tests, inspected logs/metrics/database behavior, diagnosed the cloud latency and pool-saturation issue, changed the deployment region, and verified the final production burst.

AI suggestions were treated as proposals to evaluate, not as automatically correct engineering decisions.